



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 3

Title:- free space management in os using c

Submitted by:-

Name:- swajan shrestha

Roll No:- 38

Submitted To:- Drish mali

Department Of CSIT

Checked by:-

Date of submission:- 2081/05/01

TITLE : Free space management in OS using C.

OBJECTIVE :

→ The objective of this lab is to understand and implement two fundamental methods of free space management in operating systems:

1. Bit-map,
2. Linked list

This implementation will provide insights into how operating systems manage disk space and allocate it efficiently to avoid fragmentation.

THEORY :

Free space management is a crucial aspect of an operating system's file system. It is responsible for tracking unused disk blocks to efficiently allocate and deallocate them to files as needed. Common techniques for managing free space are Bit map and Linked list.

Bit map (or bit vector) is a simple data structure where each bit represents the status of a disk block. If the bit is set to 1, the corresponding block is allocated; if it is set to 0, the block is free. Bit maps provide a compact representation, making it easy to find a free block and manage large disk spaces.

* Advantages of bit map:

- Simple to implement & understand.
- Efficient in finding contiguous free blocks.
- Compact representation, using minimal space.

disadvantages of bit-map:

↳ scanning the bit map to find a free block can be time-consuming for large disks.

In the linked list approach, all free disk blocks are linked together, forming a list. Each free block contains a pointer to the next free block in the list. When a block is allocated, it is removed from the free list and when it is freed, it is added back to the list.

* Advantages of linked list:

↳ Easy to implement and understand.

↳ Efficient in allocating and freeing the blocks, especially in dynamic environments.

* Dis-advantages of linked list:

↳ Scattered free blocks can lead to fragmentation.

↳ Traversing the list to find a block might be less efficient compared to a bit map for large disks.

OBSERVATION:

source code to implement linked list.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct SLL
```

```
{ int info;
```

```
  struct SLL* next;
```

```
};
```

```
struct SL * head = NULL;
```

```
int main()
```

```
{
```

```
    int choice, data, d;
```

```
    do
```

```
    {
```

```
        printf("\n Menu :\n");
```

```
        printf("1. Insert at beginning\n");
```

```
        printf("2. Insert at end\n");
```

```
        printf("3. Insert after a particular node\n");
```

```
        printf("4. Insert before a particular node\n");
```

```
        printf("5. Delete particular node\n");
```

```
        printf("6. Display\n");
```

```
        printf("7. Exit\n");
```

```
        printf("Enter your choice :");
```

```
        scanf("%d", &choice);
```

```
    switch (choice)
```

```
    {
```

```
        case 1:
```

```
            printf("Enter the data :");
```

```
            scanf("%d", &data);
```

```
            struct SL * ptr1 = (struct SL *) malloc (sizeof(struct  
                SL));
```

```
            ptr1->info = data;
```

```
            ptr1->next = head;
```

```
            head = ptr1;
```

```
            break;
```

```
        case 2:
```

```
            printf("Enter the data :");
```

```
            scanf("%d", &data);
```

```

struct su * ptr2 = (struct su *) malloc (sizeof (struct
su));
ptr2 → info = data;
ptr2 → next = NULL;
if (head == NULL)
{ head = ptr2; }
else
{
struct su * temp = head;
while (temp → next != NULL)
{ temp = temp → next; }
temp → next = ptr2;
}
break;

```

case 3°

```

printf ("Enter the new data to insert :");
scanf ("%d", &data);
printf ("Enter the node data after which to insert:");
scanf ("%d", &d);
struct su * ptr3 = (struct su *) malloc (sizeof (struct su));
ptr3 → info = data;
struct su * temp3 = head;
while (temp3 != NULL && temp3 → info != d)
{ temp3 = temp3 → next; }
if (temp3 == NULL)
{ printf ("Node with key %d not found \n", d);
}
else
{
ptr3 → next = temp3 → next;
temp3 → next = ptr3; }

```


break;

case 4:

printf("Enter the new data to insert");

scanf("%d", &data);

printf("Enter the node after before which to
insert");

scanf("%d", &d);

struct SL *ptr4 = (struct SL *) malloc (size of (struct
SL));

ptr4 → info = data;

struct SL *temp4 = head;

struct SL *prev4 = NULL;

while (temp4 != NULL && temp4 → info != d)

{ prev4 = temp4;

temp4 = temp4 → next;

}

if (temp4 == NULL)

{ printf("Node with key %d not found\n", d); }

else

{ if (prev4 == NULL)

{

ptr4 → next = head;

head = ptr4;

}

else

{ ptr4 → next = temp4;

prev4 → next = ptr4;

}

}

break;

Case 5 :

```
printf("Enter the node data to delete :");  
scanf("%d", &d);
```

```
if (head == NULL)
```

```
{ printf("empty linked list \n"); }
```

else

```
{ struct sll * temp5 = head;
```

```
  struct sll * prev5 = NULL;
```

```
  while (temp5 != NULL && temp5->info != d)
```

```
  { prev5 = temp5;
```

```
    temp5 = temp5->next;
```

```
  }
```

```
  if (temp5 == NULL)
```

```
  { printf("Node with key %d not found \n", d); }
```

else

```
{ if (prev5 == NULL)
```

```
  { head = head->next;
```

```
  }
```

else

```
{ prev5->next = temp5->next;
```

```
 }
```

```
free (temp5);
```

```
 }
```

```
}
```

```
break;
```

Case 6 :

```
if (head == NULL)
```

```
{ printf("list is empty \n"); }
```

```

else {
printf("Elements in the list are :");
struct sll * temp = head;
while (temp != NULL)
{
printf("%d", temp->info);
temp = temp->next;
}
printf("\n");
}
break;
case 7:
exit(0);
}
}
while (choice != 7);
return 0;
}

```

OUTPUT :

Menu :

1. Insert at beginning
2. Insert at end
3. Insert after a particular node
4. Insert before a particular node
5. Delete particular node
6. Display
7. Exit

Enter your choice : 1

enter the data : 45

Menu :

1. Insert at beginning

2. Insert at end
3. Insert after a particular node
4. Insert before a particular node
5. Delete particular node
6. Display
7. Exit

Enter your choice : 2

Enter the data : 48

menu:

1. Insert at beginning
2. Insert at end
3. Insert after a particular node
4. Insert before a particular node
5. Delete particular node
6. Display
7. Exit

Enter your choice : 3

Enter the new data to insert : 46

Enter the node after which to insert : 45

menu:

1. Insert at beginning
2. Insert at end
3. Insert after a particular node
4. Insert before a particular node
5. Delete particular node
6. Display
7. Exit

Enter your choice : 6

Elements in the list are : 45 46 48

Discussion :

→ In this lab, two different approaches to free space management were implemented and analyzed. The Bit-map method provides a compact & efficient way to track free and the allocated blocks, making it suitable for systems, where contiguous block allocation is necessary. However, its performance may degrade with the large disks due to the need to scan the bit map.

The linked list approach, on other hand, offers a simple & dynamic way to manage free space. While it avoids the need to scan through a bit map, it can lead to fragmentation, which may affect system performance over time & so on.

Conclusion :

→ This lab provided practical experience in implementing two fundamental techniques for the free space management in OS. The Bit map method, while compact, may require scanning for large disks. The linked list approach, although simple, can lead to fragmentation. Understanding these methods is essential for designing efficient file systems & optimizing disk usage in operating systems.